

Enabling Multimodal User Interactions for Genomics Visualization Creation

Qianwen Wang*, Xiao Liu*, Man Qing Liang*, Sehi L'Yi, Nils Gehlenborg

* Equal contribution

Harvard Medical School, Boston, MA, USA.

e-mail: {qianwen.wang, xliu, mliang, sehi.lyi, nils}@hms.harvard.edu.

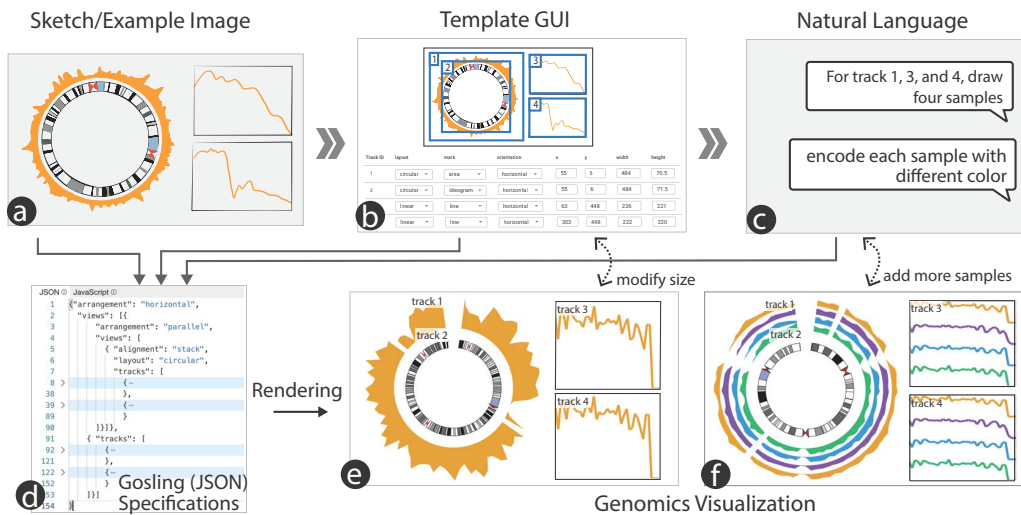


Figure 1: AutoGosling facilitates the creation of genomics visualizations by enabling multimodal interactions. Instead of directly constructing the grammar-based visualization specifications (d), users express design intentions through sketches/examples images (a), a template GUI (b), and natural language commands (c), which are interpreted by AutoGosling and converted into interactive genomics visualizations (e,f). Interactions are progressively introduced to minimize information overload and reduce mode-switching.

ABSTRACT

Visualization plays an important role in extracting insights from complex and large-scale genomics data. Traditional graphical user interfaces (GUIs) offer limited flexibility for custom visualizations. Our prior work, Gosling, enables expressive visualization creation using a grammar-based approach, but beginners may face challenges in constructing complex visualizations. To address this, we explore multimodal interactions, including sketches, example images, and natural language inputs, to streamline visualization creation. Specifically, we customize two deep learning models (YOLO v7 and GPT3.5) to interpret user interactions and convert them into Gosling specifications. A workflow is proposed to progressively introduce and integrate multimodal interactions. We then present use cases demonstrating their effectiveness and identify challenges and opportunities for future research.

Index Terms: Human-centered computing—Visualization—Visualization systems and tools; Human-centered computing—Interaction Design

1 INTRODUCTION

Advances in genomics research and high-throughput sequencing technologies have led to an explosion of complex and large-scale genomics data. Visualization plays a crucial role in understanding genomics data, revealing insights into disease-associated genetic

variants, gene expression patterns, and evolutionary relationships. Despite their significance, creating these visual representations can be challenging due to the complexity of data and the diversity of analysis needs, as surveyed by Nusrat et al. [20]

Traditionally, the creation of genomics visualizations relied on graphical user interfaces (GUIs) with predefined templates, such as IGV [32], UCSC Genome Browser [39], and JBrowse [4]. While these GUIs are instrumental for biologists without computational skills, they often limit the expressiveness in crafting custom visual representations. Some browsers support user extensions for fine-grained customization but require in-depth programming expertise (e.g., an Epilogos [19] plugin for HiGlass [10]).

Our previous work Gosling [16, 17] aims to address these challenges using a grammar-based approach. By providing primitives that map genomics data to visual properties, Gosling provides expressive building blocks to compose scalable and interactive genomics visualizations. However, because Gosling's grammar prioritizes expressiveness, sometimes it can be cumbersome to construct a visualization specification, leading to challenges in the efficient generation of complex visualizations. For example, constructing two comparative matrices with Gosling requires a JSON specification exceeding 500 lines¹. As a result, there is a need to explore solutions that streamline the visualization creation process while retaining the advantages of Gosling's expressiveness. Multimodal interactions are particularly valuable for addressing this challenge because they enable effective communication of visualization intentions [29].

In this paper, we explore the potential of multimodal interactions

¹<https://gosling.js.org/?example=MATRIX.HFFC6>

(i.e., visualization sketches, example visualization images, and natural language inputs) in facilitating the construction of Gosling’s declarative specifications in the context of genomics visualization creation. We customize two machine learning (ML) models (YOLO v7 and GPT3.5) to interpret user interactions, which are then transformed and integrated into Gosling specifications through a three-step workflow. We present a use case that exemplifies how multimodal interactions can enhance the user experience and highlight the challenges and opportunities associated with integrating multimodal interactions into genomics visualization tools.

2 RELATED WORK

2.1 Diverse Ways for Creating Data Visualization

Grammar-based: Visualization grammars empower users to create flexible and expressive visualizations through declarative specifications. Previous studies have proposed diverse visualization grammars [18], influencing both general-purpose visualizations (e.g., Vega-Lite [28], ggplot2 [35]) and specialized visualizations (e.g., Gosling for genomics visualization [16], ATOM for unit visualization [23]). While these grammars offer a powerful means of visualization creation, they usually have steep learning curves and pose challenges for users lacking programming skills.

GUI-based: To enable visualization creation without programming, systems have been proposed to help users create visualization through a graphic user interface (GUI). Satyanarayan et al. [27] categorized these GUIs into three groups: 1) template editors, such as MS Excel and IGV [32], which leverage predefined templates for straightforward creation; (2) shelf construction tools, such as Polaris [30] and Voyager [36], which map data fields to visual channels; and (3) visual builders, such as Data-Driven Guides [11], Lyra [26], and Data Illustrator [14], which provide granular control over marks and layouts. However, GUIs can be complex for expressive visualizations, posing navigational challenges.

Natural-Interaction-Based: Through natural interactions, such as text [8, 24, 38] and sketches [3, 31], users can intuitively create a wide range of visualizations without being constrained by predefined templates. However, it can be challenging to accurately describe complex visualizations using only one modality, such as an extensive textual description, due to the intricacy of the visual information. Therefore, recent studies [12, 29] have explored the integration of multimodal interactions (e.g., pen, touch, and speech in InChorus [29]). Nevertheless, these studies typically focus on general visualizations and provided limited discussion about crafting complex, multi-mark, multi-view genomics visualizations.

To address this gap, AutoGosling assists users in constructing declarative specifications of genomics visualizations (Gosling [16] in our implementation) through the integration of sketching, example images, shelf construction, and natural language interactions.

2.2 Challenges in Visualizing Genomics Data

In data mapped to a genome sequence (~3 billion nucleotides in the human reference genome), patterns of interest can arise at multiple regions and scales, from multiple datasets [16]. This often requires complex arrangements of many views with coordinated interactions for effective visual analytics [15]. Many genomics visualization tools, including genome browsers [5, 10, 32], provide visualization templates with little customization options, putting more emphasis on ease of use over expressiveness. On the other hand, lower-level visualization libraries, such as the Gosling visualization grammar [16, 17], Circos [13], and ggBio [37], provide much customization flexibility, but often make it difficult for biologists without computational skills to learn and construct visualizations quickly. Several genomics tools provide rule-based visualization recommendations to broaden their target audience [22, 37]. For example, GenoREC [22] recommends visualizations for given genomics datasets and tasks and exports corresponding Gosling specifications

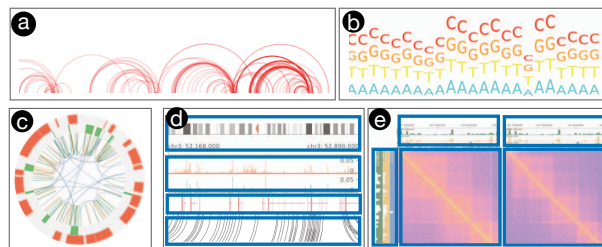


Figure 2: Examples in the training dataset. The dataset encompasses diverse genomics visualizations with varied mark types (a–e), track orientations (a–e), and view configurations (d–e).

```
view := tracks, layout?, alignment?, orientation?,
assembly?, linkingId?, ...
tracks := (track | view)*
layout := "linear" | "circular"
orientation := "horizontal" | "vertical"
track := data, dataTransform?, mark, x?, y?, color?,
size?, displacement?, visibility?, ...
mark := "point" | "line" | "bar" | "link" | ...
```

Figure 3: Key components in a Gosling visualization specification that are detected in AutoGosling.

for further customization (e.g., changing colors or sizes). However, customizing the extensive JSON specifications is still challenging for users.

In our paper, we adopt modalities that are novel in genomics visualization construction, such as sketches and natural language, to enable the efficient and effective creation of expressive genomics visualizations.

3 DESIGN GOALS

D1. Simplify genomics visualization specification: Gosling employs a state-of-the-art grammar-based approach to support expressive genomics visualization creation. Despite its remarkable expressiveness, Gosling’s grammar can lead to lengthy and complex JSON specifications for multi-view glyph-based genomic visualizations. By introducing multimodal interactions, we enable users to easily convey their intentions for the visualization into the corresponding Gosling JSON specifications.

D2. Combine the advantages of different modes of interactions. Each mode of interaction offers unique advantages. For example, sketching allows users to quickly convey the desired visualization layout, while natural language input enables users to specify details. Multimodal interfaces can leverage the strengths of each mode to provide a more comprehensive and flexible approach to genomics visualization creation. The interface and the workflow should be designed to fully leverage the strengths of each mode.

D3. Minimize confusion with interaction modes: Constantly switching between interaction modes for the same input can be disruptive to the users’ workflow [29]. Therefore, the interface should introduce these interactions progressively [34], ensuring that users can easily navigate between different modes of interaction.

4 SYSTEM

This section describes how AutoGosling supports and integrates different modes of user interactions to facilitate the creation of genomics visualization (i.e., the construction and modification of a Gosling visualization specification).

4.1 A Workflow to Integrate User Interactions

AutoGosling’s interface features a streamlined three-step workflow that progressively introduces interactions, minimizing information overload and reducing the need for frequent mode-switching. Each step thoughtfully incorporates interactions that maximize the unique benefits of each input mode.

Step 1: Ideation via sketch and example images

People commonly use sketches [25] and example images [1, 7] to communicate visualization design intentions. Sketches and example images offer distinct advantages over other forms of interactions. For instance, while users may need to compose a lengthy text to accurately convey their design intentions (e.g., view layout, chart types), a single image can easily communicate the same information.

Users upload an image of a visualization example or a sketch. The design intentions conveyed by the image will be interpreted by an object detection model.

Step 2: Modification through template GUI

The outputs of Step 1 is used to reconstruct Gosling visualization specifications based on a set of predefined rules. As shown in Fig. 5, the left side shows the detection results and the right side correspondingly presents the reconstructed Gosling visualizations.

Recognizing that image interpretation may not always be flawless, AutoGosling provides users with the capability to refine the interpreted design intentions through a template GUI. As users make modifications, the reconstructed Gosling visualizations are updated in real-time for immediate feedback.

To facilitate this process, AutoGosling assigns a unique ID to each track and presents the information associated with each track in an interactive table, where each row corresponds to a specific track. When users click on and edit a row, the corresponding track is highlighted for easy reference. By presenting track information in an interactive table rather than overlaying it directly on the image, AutoGosling effectively avoids visual clutter.

Step 3: Customization with natural language

Interpreting design intentions via images may miss some information by default, either because they are not depicted in sketch images or because they fall outside the scope of detection. Common examples of such information include the specified range of chromosomes, the connections linking two views, and the source of datasets. Additionally, users may wish to explore design alternatives, such as experimenting with different color schemes for visual marks.

While users can directly modify the Gosling JSON specification generated from Step 2, navigating and identifying the appropriate section for the desired modification can be challenging, especially for expressive visualizations, whose specifications are usually lengthy, complex, and hierarchical.

Therefore, AutoGosling enables users to add and modify details via natural language commands. Users input text that describes the desired modification. The Gosling specification and visualization will then be updated accordingly. If the generated Gosling specification contains any syntax errors, the Gosling compiler will identify the sections associated with the error and provide helpful hints for resolution. Users can manually correct the errors based on the hints or submit the logs back to AutoGosling, which will then update the specifications accordingly.

4.2 Techniques to support user interactions

ML models have demonstrated great performance in interpreting natural interactions. However, to effectively utilize these techniques in visualization creation, customization and fine-tuning are essential [33]. Please refer to our project page <https://github.com/autogosing/autogosing-tool> for more technical details about the model fine-tuning and the prompt-crafting.

Interpret Design Intentions via Sketch and Example Images.

An object detection model is used to interpret users’ design intentions in the sketches and example images. We chose and fine-tuned

the YOLO v7 model as it is the best real-time object detection algorithm based on the Average Precision (AP) on the MS COCO dataset, as of Feb 2023 [2]. The model is pre-trained on the MS COCO dataset and then fine-tuned using a genomics visualization dataset. This fine-tuning dataset consisted of in total 40,470 examples and was created by systematically varying a range of design options, such as the types and colors of the marks, the track layout, and the arrangement of views, using the example gallery of Gosling as a basis. Compared with general visualization datasets such as VIS30k [6], our dataset focuses more on the design intentions of multi-view genomics visualizations. Outputs of the object detection model are a set of bounding boxes with class labels. Each box represents a track, an essential component of genomics visualization that shows genomics features or data aligned along a linear representation of the genome [20]. A box describes the location, width, and height of the track, while the accompanying class labels describe the specific characteristics of the track, such as the mark types (e.g., area, line, point) and the track layout (linear or circular). Fig. 3 summarizes the key components in the Gosling visualization specifications. Table 1 shows that the fine-tuned model is able to generate accurate labels for the genomics visualization images. Because of the shared visual elements (i.e., geometric shapes) and patterns (i.e., view arrangement) between sketches and visualization images, the model is also able to generate reliable predictions for sketch images.

Class	Labels	P	R	mAP@.5	mAP@.5:.95
all	50420	0.995	0.967	0.976	0.975
layout	16512	0.997	0.975	0.994	0.993
orientation	16512	1	0.974	0.992	0.989
marks	17396	0.994	0.97	0.982	0.981

Table 1: Performance of the detection model when testing on a hold-out set of 8069 images. P: precision, R: recall.

Generate Gosling Specifications

To construct a Gosling specification from the object detection results, we first merge the bounding boxes and corresponding labels for each track based on ratio of intersections (threshold = 0.75). Then for each detected track, we construct a Gosling specification using the associated labels (i.e., mark types, track orientation, and view layout) and the size of the bounding box (i.e., width and height for the track). Finally, we use a bottom-top approach to recursively assemble these tracks into views based on their positions.

Given that the input images may lack certain information (e.g., dataset, the domain of the chromosome), and that the object detection model may miss specific details (e.g., color of the mark outline), we use representative values as default. For example, for the datasets, we curated a collection of representative genomics datasets and selected specific datasets based on the mark types, taking into consideration the recommended visualizations for various genomics datasets as outlined by Pandey et al [22].

```
{ 'role': 'system', 'content': 'You're an assistant designed to edit Gosling specification JSON code.' },
{ 'role': 'user', 'content': [Domain.Instructions] },
{ 'role': 'user', 'content': 'modify Gosling Specification [Current.Spec]' },
{ 'role': 'user', 'content': [User.Input] }
```

Figure 4: The prompt template. Domain.Instructions contains a list of messages designed to correct common errors in GPT-generated Gosling specifications.

Modify Gosling Specifications via Natural Language.

To support the editing of Gosling visualization specifications via natural language, a GPT-3.5 model (“gpt-3.5-turbo-0301”) is customized using a set of domain-specific prompts. Even though the GPT-3.5 model has demonstrated great performance in a range



Figure 5: AutoGosling identifies the design intentions presented in a sketch image (A) and transforms them into an interactive genomics visualization. Users can further customize and refine the visualization using predefined templates (B) and natural language commands.

of programming and visualization tasks, our experiments showed that it has a limited understanding of the domain-specific Gosling’s grammar. For example, by default, the GPT-generated specifications did not include ‘track’, which is an essential component of genomics visualizations that shows genomics features or data aligned along a linear representation of the genome [16, 20]. As a result, we summarized the typical errors and fixed them using domain-specific instructions. These domain-specific instructions can be categorized into three groups: data-, encoding-, and view-related.

- Data-related instructions outline the process of loading genomics datasets in Gosling.
- Encoding-related instructions provide guidance on how Gosling maps data values to visual channels, such as “*there must be one genomics axis*”.
- View-related instructions describe the configuration of interactions within and between views in Gosling, such as “*tracks in the same view will be linked*”.

As shown in Fig. 4, we added these domain-specific introductions as user messages since GPT-3.5 does not pay strong attention to system messages [21].

5 USE CASE

This section presents two use-case scenarios to illustrate how AutoGosling enables a fictional biologist, Lilly, to create interactive genomics visualizations. Sketches in the use cases (Fig. 1, Fig. 5) were drawn by an author using the Notability app on an iPad.

Multi-Track: Lilly has three BigWig files [9] that she wants to identify correlations between them. Lilly has ideas of how each data should be visually encoded, so she draws a sketch of three tracks using an iPad and uploads it to AutoGosling (Fig. 5A). AutoGosling automatically analyzes the design intention conveyed in the uploaded image, such as layouts, mark types, and size of tracks. However, due to inaccurate sketches, Lilly finds that the tracks have inconsistent widths (Fig. 5B). Therefore, Lilly manually adjusts the size of tracks and moves to the next step, where she further modifies the track color by inputting text “*use different colors for tracks*”. Lilly is satisfied with the visual encoding, so she applies the visualization to her own dataset by inputting text “*use the data from this URL*”.

Overview + Detail: Gosling’s default zoom interactions facilitate the identification of important visual patterns at varying scales. However, users often need overviews to contextualize their investigation of local details, which can be cumbersome to create when writing JSON specifications. To achieve this easily, Lilly uploads her sketch to AutoGosling that depicts a combination of an overview and two detailed views (Fig. 1a). The left side of the sketch features an ideogram track with an area chart track, serving as an overview. In contrast, the right side includes two line chart tracks, showing local genomic regions. AutoGosling interprets this sketch and converts it into a Gosling specification (Fig. 1d), which is used to generate interactive visualizations on AutoGosling (Fig. 1e). As Lilly interacts with the generated visualization directly on AutoGosling, she decides to visualize multiple samples within each track, instead of a single sample. To achieve this, she inputs the text “*for track 1, 3, and 4, draw four samples and encode each sample with different color*”, expressing her intention to improve the visualization further.

6 DISCUSSION

Challenges in Applying ML for VIS: While our use cases demonstrate the effectiveness of our approach, we also observed certain limitations associated with the customized AI models. First, the object detection model is only capable of detecting a predefined set of information. Therefore, some design intentions will be missed. Additionally, while the model can effectively identify digital sketches, it exhibits reduced performance when interpreting pen-and-paper sketches, which often look considerably different from the training dataset. Second, the GPT model faces challenges when tasked with modifying highly hierarchical JSON specifications. Additionally, the model encounters difficulties when asked to load external genomics datasets, as it lacks the capability to read the structure and content of such datasets.

Limitations in Evaluation: First, we primarily relied on use case demonstrations to showcase the effectiveness of our approach. To further validate the practicality of our tool, we plan to conduct user studies involving domain experts. These studies will provide valuable insights into how experts interact with and respond to multimodal user interactions. Second, we did not conduct quantitative evaluation for natural-language interactions. Currently, there is no established benchmark or metrics for evaluating natural language instructions in the context of visualization creation, which is crucial for improving the tool’s interpretive capabilities.

7 CONCLUSION

In this paper, we explored the potential of multimodal interactions to streamline the visualization creation process and enhance user experience. We employed and adapted state-of-the-art deep learning models, including YOLO v7 for object detection and GPT-3.5 for natural language processing, to interpret user inputs in the form of sketches, example images, and textual descriptions. A three-step workflow progressively incorporates multimodal interactions and guides users through the stages of ideation, refinement, and finalization. Two use cases demonstrate the method’s effectiveness, and we highlight opportunities for future multimodal genomics visualization research, including the infusion of visualization knowledge into general AI models and implementation of benchmark evaluations on genomics visualization.

As genomics research continues to evolve, we believe that the integration of multimodal interactions will play a pivotal role in empowering users to effectively communicate their visualization intentions and harness the full potential of genomics data.

ACKNOWLEDGMENTS

This work was supported by the National Institutes of Health (R01HG011773).

REFERENCES

- [1] H. K. Bako, X. Liu, L. Battle, and Z. Liu. Understanding how designers find and use data visualization examples. *IEEE Transactions on Visualization and Computer Graphics*, 29(1):1048–1058, 2022.
- [2] G. Boesch. Object detection in 2023: The definitive guide, 2023.
- [3] J. Browne, B. Lee, S. Carpendale, N. Riche, and T. Sherwood. Data analysis on interactive whiteboards through sketch-based interaction. In *Proceedings of the ACM International Conference on Interactive Tabletops and Surfaces*, pp. 154–157, 2011.
- [4] R. Buels, E. Yao, C. M. Diesh, R. D. Hayes, M. Munoz-Torres, G. Helt, D. M. Goodstein, C. G. Elisk, et al. JBrowse: a dynamic web platform for genome visualization and analysis. *Genome Biology*, 17(1):1–12, 2016.
- [5] F. Chelaru, L. Smith, N. Goldstein, and H. C. Bravo. Epiviz: interactive visual analytics for functional genomics data. *Nature Methods*, 11(9):938–940, 2014.
- [6] J. Chen, M. Ling, R. Li, P. Isenberg, T. Isenberg, M. Sedlmair, T. Möller, R. S. Laramée, H.-W. Shen, K. Wünsche, and Q. Wang. Vis30k: A collection of figures and tables from iee visualization conference publications. *IEEE Transactions on Visualization and Computer Graphics*, 27:3826–3833, 2020.
- [7] Z. Chen, Y. Wang, Q. Wang, Y. Wang, and H. Qu. Towards automated infographic design: Deep learning-based auto-extraction of extensible timeline. *IEEE Transactions on Visualization and Computer Graphics*, 26(1):917–926, 2020. doi: 10.1109/TVCG.2019.2934810
- [8] W. Cui, X. Zhang, Y. Wang, H. Huang, B. Chen, L. Fang, H. Zhang, J.-G. Lou, and D. Zhang. Text-to-viz: Automatic generation of infographics from proportion-related natural language statements. *IEEE Transactions on Visualization and Computer Graphics*, 26(1):906–916, 2019.
- [9] W. J. Kent, A. S. Zweig, G. Barber, A. S. Hinrichs, and D. Karolchik. Bigwig and bigbed: enabling browsing of large distributed datasets. *Bioinformatics*, 26(17):2204–2207, 2010.
- [10] P. Kerpedjiev, N. Abdennur, F. Lekschas, C. McCallum, K. Dinkla, H. Strobel, J. M. Luber, S. B. Ouellette, A. Azhir, N. Kumar, et al. Higglass: web-based visual exploration and analysis of genome interaction maps. *Genome Biology*, 19(1):1–12, 2018.
- [11] N. W. Kim, E. Schweickart, Z. Liu, M. Dontcheva, W. Li, J. Popovic, and H. Pfister. Data-driven guides: Supporting expressive design for information graphics. *IEEE Transactions on Visualization and Computer Graphics*, 23(1):491–500, 2016.
- [12] Y.-H. Kim, B. Lee, A. Srinivasan, and E. K. Choe. Data@hand: Fostering visual exploration of personal data on smartphones leveraging speech and touch interaction. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, 2021.
- [13] M. Krzywinski, J. Schein, I. Birol, J. Connors, R. Gascoyne, D. Horsman, S. J. Jones, and M. A. Marra. Circos: an information aesthetic for comparative genomics. *Genome Research*, 19(9):1639–1645, 2009.
- [14] Z. Liu, J. Thompson, A. Wilson, M. Dontcheva, J. Delorey, S. Grigg, B. Kerr, and J. Stasko. Data illustrator: Augmenting vector design tools with lazy data binding for expressive visualization authoring. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, pp. 1–13, 2018.
- [15] S. L’Yi and N. Gehlenborg. Multi-view design patterns and responsive visualization for genomics data. *IEEE Transactions on Visualization and Computer Graphics*, 29(1):559–569, 2022.
- [16] S. L’Yi, Q. Wang, F. Lekschas, and N. Gehlenborg. Gosling: A Grammar-based Toolkit for Scalable and Interactive Genomics Data Visualization. *IEEE Transactions on Visualization and Computer Graphics*, (1):140–150, 1 2022. doi: 10.1109/TVCG.2021.3114876
- [17] T. Manz, S. L’Yi, and N. Gehlenborg. Gos: a declarative library for interactive genomics visualization in python. *Bioinformatics*, 39(1):btad050, 2023.
- [18] A. M. McNutt. No grammar to rule them all: A survey of json-style dsls for visualization. *IEEE Transactions on Visualization and Computer Graphics*, 29(1):160–170, 2023.
- [19] W. Meuleman, A. Reynolds, E. Rynes, and C. Lundberg. Epilogos: information-theoretic navigation of multi-tissue functional genomic annotations, 2021. <https://epilogos.altius.org/>.
- [20] S. Nurat, T. Harbig, and N. Gehlenborg. Tasks, techniques, and tools for genomic data visualization. *Computer Graphics Forum*, 38(3):781–805, 2019.
- [21] OpenAI. Instructing chat models, 2023.
- [22] A. Pandey, S. L’Yi, Q. Wang, M. A. Borkin, and N. Gehlenborg. Genorec: A recommendation system for interactive genomics data visualization. *IEEE Transactions on Visualization and Computer Graphics*, 29:570 – 580, 2022.
- [23] D. Park, S. M. Drucker, R. Fernandez, and N. Elmqvist. Atom: A grammar for unit visualizations. *IEEE Transactions on Visualization and Computer Graphics*, 24(12):3032–3043, 2017.
- [24] M. M. Rashid, H. K. Jahan, A. Huzzat, R. A. Rahul, T. B. Zakir, F. Meem, M. S. H. Mukta, and S. Shatabda. Text2chart: A multi-staged chart generator from natural language text. In *Advances in Knowledge Discovery and Data Mining: 26th Pacific-Asia Conference, PAKDD 2022, Chengdu, China, May 16–19, 2022, Proceedings, Part II*, pp. 3–16. Springer, 2022.
- [25] J. C. Roberts, C. Headleand, and P. D. Ritsos. Sketching designs using the five design-sheet methodology. *IEEE Transactions on Visualization and Computer Graphics*, 22(1):419–428, 2015.
- [26] A. Satyanarayan and J. Heer. Lyra: An interactive visualization design environment. In *Computer Graphics Forum*, vol. 33, pp. 351–360. Wiley Online Library, 2014.
- [27] A. Satyanarayan, B. Lee, D. Ren, J. Heer, J. Stasko, J. Thompson, M. Brehmer, and Z. Liu. Critical reflections on visualization authoring systems. *IEEE Transactions on Visualization and Computer Graphics*, 26(1):461–471, 2020. doi: 10.1109/TVCG.2019.2934281
- [28] A. Satyanarayan, D. Moritz, K. Wongsuphasawat, and J. Heer. Vega-lite: A grammar of interactive graphics. *IEEE Transactions on Visualization and Computer Graphics*, 23(1):341–350, 2017.
- [29] A. Srinivasan, B. Lee, N. Henry Riche, S. M. Drucker, and K. Hinckley. Inchorus: Designing consistent multimodal interactions for data visualization on tablet devices. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*, pp. 1–13, 2020.
- [30] C. Stolte, D. Tang, and P. Hanrahan. Polaris: A system for query, analysis, and visualization of multidimensional relational databases. *IEEE Transactions on Visualization and Computer Graphics*, 8(1):52–65, 2002.
- [31] Z. Teng, Q. Fu, J. White, and D. C. Schmidt. Sketch2vis: Generating data visualizations from hand-drawn sketches with deep learning. In *2021 20th IEEE International Conference on Machine Learning and Applications (ICMLA)*, pp. 853–858. IEEE, 2021.
- [32] H. Thorvaldsdóttir, J. T. Robinson, and J. P. Mesirov. Integrative genomics viewer (igv): high-performance genomics data visualization and exploration. *Briefings in Bioinformatics*, 14(2):178–192, 2013.
- [33] Q. Wang, Z. Chen, Y. Wang, and H. Qu. A survey on ml4vis: Applying machine learning advances to data visualization. *IEEE Transactions on Visualization and Computer Graphics*, 28(12):5134–5153, 2021.
- [34] Q. Wang, Z. Li, S. Fu, W. Cui, and H. Qu. Narvis: Authoring narrative slideshows for introducing data visualization designs. *IEEE Transactions on Visualization and Computer Graphics*, 25(1):779–788, 2019.
- [35] H. Wickham. ggplot2. *Wiley Interdisciplinary Reviews: Computational Statistics*, 3(2):180–185, 2011.
- [36] K. Wongsuphasawat, D. Moritz, A. Anand, J. Mackinlay, B. Howe, and J. Heer. Voyager: Exploratory analysis via faceted browsing of visualization recommendations. *IEEE Transactions on Visualization and Computer Graphics*, 22(1):649–658, 2015.
- [37] T. Yin, D. Cook, and M. Lawrence. ggbio: an r package for extending the grammar of graphics for genomic data. *Genome Biology*, 13(8):1–14, 2012.
- [38] B. Yu and C. T. Silva. Flowsense: A natural language interface for visual data exploration within a dataflow system. *IEEE Transactions on Visualization and Computer Graphics*, 26(1):1–11, 2019.
- [39] J. Zhu, J. Z. Sanborn, S. Benz, C. Szeto, F. Hsu, R. M. Kuhn, D. Karolchik, J. Archie, M. E. Lenburg, L. J. Esserman, et al. The ucsc cancer genomics browser. *Nature Methods*, 6(4):239–240, 2009.